

Local Binding

Substitution

CMSC 323: Design and Implementation of Programming Languages

This lecture note is adapted from content created by Robby Findler (Northwestern University) and Vincent St-Amour (Northwestern University) and Shriram Krishnamurthi in *Programming Languages: Application and Interpretation*.

With + Arithmetic Language

$\langle \text{WAE} \rangle ::= \langle \text{num} \rangle$

| $\{ + \langle \text{WAE} \rangle \langle \text{WAE} \rangle \}$

| $\{ - \langle \text{WAE} \rangle \langle \text{WAE} \rangle \}$

| $\{ \text{with } \{ \langle \text{var} \rangle \langle \text{WAE} \rangle \} \langle \text{WAE} \rangle \}$

| $\langle \text{var} \rangle$

What are we doing now?

$\{ \text{with } \{ x \{ + 1 2 \} \}$

$\{ + x x \} \}$

$\Rightarrow 6$

With + Arithmetic Language

$\langle \text{WAE} \rangle ::= \langle \text{num} \rangle$
| $\{ + \langle \text{WAE} \rangle \langle \text{WAE} \rangle \}$
| $\{ - \langle \text{WAE} \rangle \langle \text{WAE} \rangle \}$
| $\{ \text{with } \{ \langle \text{var} \rangle \langle \text{WAE} \rangle \} \langle \text{WAE} \rangle \}$
| $\langle \text{var} \rangle$

Our program

X

$\Rightarrow$

Error: free identifier

With + Arithmetic Language

```
<WAE> ::= <num>
| {+ <WAE> <WAE>}
| {- <WAE> <WAE>}
| {with {<var> <WAE>} <WAE>}
| <var>
```

```
{+      {with {x {+ 1 2}}
          {+ x x}}}
{with {y {- 4 3}}
      {+ y y}}}
```

=> 8

With + Arithmetic Language

```
<WAE> ::= <num>
        | {+ <WAE> <WAE>}
        | {- <WAE> <WAE>}
        | {with {<var> <WAE>} <WAE>}
        | <var>
```

```
{with {x {+ 1 2}}
      {with {x {- 4 3}}
        {+ x x}}}}
```

=> 2

With + Arithmetic Language

```
<WAE> ::= <num>
        | {+ <WAE> <WAE>}
        | {- <WAE> <WAE>}
        | {with {<var> <WAE>} <WAE>}
        | <var>
```

```
{with {x {+ 1 2}}
      {with {y {- 4 3}}
        {+ x x}}}}
```

=> 6

Implementing Local Bindings

We implement local binding in two ways:

1. Substitution
2. Deferred/Cached Substitution

With + Arithmetic Language

Extending our PLAIT data type to include our new surface syntax

```
<WAE> ::= <num>
        | {+ <WAE> <WAE>}
        | {- <WAE> <WAE>}
        | {with {<var> <WAE>} <WAE>}
        | <var>
```

```
(define-type WAE
  [num (n : Number)]
  [plus (left : WAE)
        (right : WAE)]
  [sub (left : WAE)
        (right : WAE)]
  [with (id : Symbol)
        (value : WAE)
        (body : WAE)]
  [var (s : Symbol)])
```

How Substitution Works

Example 1

First **evaluate the expression** to be associated with the variable

$$\{\text{with } \{x \{+ 1 2\}\} \{+ x x\}\} \Rightarrow \{\text{with } \{x 3\} \{+ x x\}\}$$

Evaluating the value to be associated with the variable **before** assigning it, is
called **eager evaluation**

How Substitution Works

Example 1

Then **substitute the evaluated value** in the body of the 'with'

$$\begin{aligned} \{\text{with } \{x \{+ 1 2\}\} \\ \{+ x x\}\} &\Rightarrow \{\text{with } \{x 3\} \\ \{+ x x\}\} &\Rightarrow \{\text{with } \{x 3\} \\ \{+ 3 3\}\} \end{aligned}$$

How Substitution Works

Example 1

Finally interpret the body of the with

{with {x {+ 1 2}}
 {+ x x}}

{with {x 3}
 {+ x x}}

{with {x 3}
 {+ 3 3}}

6

How Substitution Works

Example 2

<code>{+ {with {x {+ 1 2}} {+ x x}} {with {y {- 4 3}} {+ y y}}}}</code>	<code>{+ {with {x 3} {+ x x}} {with {y 1} {+ y y}}}}</code>	<code>{+ {with {x 3} {+ 3 3}} {with {y 1} {+ 1 1}}}}</code>	<code>{+ {+ 3 3} {+ 1 1}}</code>	<code>=> 8</code>
---	---	---	--	----------------------

Eager-evaluation:

Evaluate (interpret) the
value to be assigned

substitution:

Substitute x and
y in their
respective
scopes

interpret:

Evaluate the
bodies of the
withs

How Substitution Works

Example 3

$\{+ \{with \{x \{+ 1 2\}\} \{+ x x\}\} \{with \{x \{- 4 3\}\} \{+ x x\}\}\}$	$\{+ \{with \{x 3\} \{+ x x\}\} \{with \{x 1\} \{+ x x\}\}\}$	$\{+ \{with \{x 3\} \{+ 3 3\}\} \{with \{x 1\} \{+ 1 1\}\}\}$	$\{+ \{+ 3 3\} \{+ 1 1\}\} \Rightarrow 8$
---	---	---	---

Eager-evaluation:

Evaluate (interpret) the
value to be assigned

substitution:

Substitute x and y in
their respective scopes

interpret:

Evaluate the bodies
of the withs

How Substitution Works

Example 4

```
{with {x {+ 1 2}}  
  {with {x {- 4 3}}  
    {+ x x}}}
```

```
{with {x 3}  
  {with {x 1}  
    {+ x x}}}
```

```
{with {x 3}  
  {with {x 1}  
    {+ 1 1}}}  {+ 1 1}  => 2
```

Eager-evaluation:

Evaluate (interpret)

the value to be

assigned

substitution:

Substitute x and y in

their respective

scopes

The second x

shadows the first x

interpret:

Evaluate the

bodies of the withs

How Substitution Works

Example 4

```
{with {x {+ 1 2}}  
  {with {y {- 4 3}}  
    {+ x x}}}
```

```
{with {x 3}  
  {with {y 1}  
    {+ x x}}}
```

```
{with {x 3}  
  {with {y 1}  
    {+ 3 3}}}
```

{+ 3 3} => 6

Eager-evaluation:

Evaluate (interpret) the
value to be assigned

substitution:

Substitute x and y in their
respective scopes

interpret:

Evaluate the
bodies of the withs

How Substitution Works

Example 5

X

No binding occurrence found for this reference

Implementing "with" In Our Interpreter

WAE - "with + Arithmetic" Language

```
<WAE> ::= <num>
        | {+ <WAE> <WAE>}
        | {- <WAE> <WAE>}
        | {with {<var> <WAE>} <WAE>}
        | <var>
```

```
(interp : (WAE -> Number))
(define (interp wae)
  (type-case WAE wae
    [(num n) n]
    [(plus lhs rhs) (+ (interp lhs) (interp rhs))]
    [(sub lhs rhs) (- (interp lhs) (interp rhs))]
    [(with id value body) . . .]
    [(var id) . . .])))
```

Implementing "with" In Our Interpreter

First **evaluate the expression** to be associated with id.

```
<WAE> ::= <num>
        | {+ <WAE> <WAE>}
        | {- <WAE> <WAE>}
        | {with {<var> <WAE>} <WAE>}
        | <var>
```

```
(interp : (WAE -> Number))
(define (interp wae)
  (type-case WAE wae
    [(num n) n]
    [(plus lhs rhs) (+ (interp lhs) (interp rhs))]
    [(sub lhs rhs) (- (interp lhs) (interp rhs))]
    [(with id value body) . . . (interp value) . . . )])
  [(var id) ...]))
```

Implementing "with" In Our Interpreter

Then **substitute the evaluated value** in the body of the 'with'

Substitute every occurrence of id in body with the result from evaluating value

```
(interp : (WAE -> Number))  
(define (interp wae)  
  (type-case WAE wae  
    [(num n) n]  
    [(plus lhs rhs) (+ (interp lhs) (interp rhs))]  
    [(sub lhs rhs) (- (interp lhs) (interp rhs))]  
    [(with id value body) . . . (subst id (interp value) body) . . . )])  
    [(var id) . . . ]))
```

Implementing "with" In Our Interpreter

Finally **interpret the body of the with**

Interpret the body that now has the substituted values.

```
(interp : (WAE -> Number))  
(define (interp wae)  
  (type-case WAE wae  
    [(num n) n]  
    [(plus lhs rhs) (+ (interp lhs) (interp rhs))]  
    [(sub lhs rhs) (- (interp lhs) (interp rhs))]  
    [(with id value body) (interp (subst id (interp value) body)))]  
    [(var id) . . .]))
```

Substitution: Implementing Subst

```
(subst : (Symbol Number WAE -> WAE))  
(define (subst id val wae)  
  (type-case WAE wae  
    [(num n) . . .]  
    [(sub lhs rhs) . . .]  
    [(plus lhs rhs) . . .]  
    [(var my-id) . . .]  
    [(with my-id value body) . . .])))
```

Example Substitutions

`;10 for x in {+ 1 x}`

`;10 for x in x`

`;10 for x in y`

`; 10 for y in {- x 1}`

Example Substitutions

`;10 for x in {+ 1 x} => {+ 1 10}`

Example Substitutions

`;10 for x in {+ 1 x} => {+ 1 10}`

```
(test (subst 'x 10 (plus (num 1) (var 'x)))  
      (plus (num 1) (num 10)))
```

Example Substitutions

`;10 for x in {+ 1 x} => {+ 1 10}`

`(test (subst 'x 10 (plus (num 1) (var 'x)))
 (plus (num 1) (num 10)))`

`;10 for x in x => 10`

Example Substitutions

```
;10 for x in {+ 1 x} => {+ 1 10}  
(test (subst 'x 10 (plus (num 1) (var 'x)))  
      (plus (num 1) (num 10)))
```

```
;10 for x in x => 10  
(test (subst 'x 10 (var 'x))  
      (num 10))
```

```
;10 for x in y => y
```

Example Substitutions

```
;10 for x in {+ 1 x} => {+ 1 10}  
(test (subst 'x 10 (plus (num 1) (var 'x)))  
      (plus (num 1) (num 10)))
```

```
;10 for x in x => 10  
(test (subst 'x 10 (var 'x))  
      (num 10))
```

```
;10 for x in y => y  
(test (subst 'x 10 (var 'y) )  
      (var 'y))
```

```
; 10 for y in {- x 1} => {- x 1}
```

Example Substitutions

```
;10 for x in {+ 1 x} => {+ 1 10}  
(test (subst 'x 10 (plus (num 1) (var 'x)))  
      (plus (num 1) (num 10)))
```

```
;10 for x in x => 10  
(test (subst 'x 10 (var 'x))  
      (num 10))
```

```
;10 for x in y => y  
(test (subst 'x 10 (var 'y) )  
      (var 'y))
```

```
; 10 for y in {- x 1} => {- x 1}  
(test (subst 'y 10 (sub (var 'x) (num 1)))  
      (sub (var 'x) (num 1)))
```

Substitution

```
(subst : (Symbol Number WAE -> WAE))  
(define (subst id val wae)  
  (type-case WAE wae  
    [(num n) wae]  
    [(plus lhs rhs) (plus (subst id val lhs)  
                           (subst id val rhs))]  
    [(sub lhs rhs) (sub (subst id val lhs)  
                        (subst id val rhs))]  
    [(var my-id) (if (eq? id my-id) (num val) wae)]  
    [(with my-id value body) . . .])))
```

More Examples

```
; 10 for x in {with {y 17} x} => {with {y 17} 10}  
(test (subst 'x 10 (with 'y (num 17) (var 'x)))  
      (with 'y (num 17) (num 10)))
```

```
; 10 for x in {with {y x} y} => {with {y 10} y}  
(test (subst 'x 10 (with 'y (var 'x) (var 'y)))  
      (with 'y (num 10) (var 'y)))
```

```
; 10 for x in {with {x y} x} => {with {x y} x}  
(test (subst 'x 10 (with 'x (var 'y) (var 'x)))  
      (with 'x (var 'y) (var 'x)))
```

Substitution

```
(subst : (Symbol Number WAE -> WAE))
(define (subst id val wae)
  (type-case WAE wae
    [(num n) wae]
    [(plus lhs rhs) (plus (subst id val lhs)
                           (subst id val rhs))]
    [(sub lhs rhs) (sub (subst id val lhs)
                        (subst id val rhs))]
    [(var my-id) (if (eq? id my-id) (num val) wae)]
    [(with my-id value body) (if (eq? id my-id)
                                  (with my-id (subst id val value) body)
                                  (with my-id (subst id val value)
                                              (subst id val body)))]))
```

What if the body of the
"with" is also a "with"?

Implementing "with" In Our Interpreter

If a variable reference is not already substituted by the time it is to be interpreted, it is a free id

```
(interp : (WAE -> Number))  
(define (interp wae)  
  (type-case WAE wae  
    [(num n) n]  
    [(plus lhs rhs) (+ (interp lhs) (interp rhs))]  
    [(sub lhs rhs) (- (interp lhs) (interp rhs))]  
    [(with id value body) (interp (subst id (interp value) body)))]  
    [(var id) (error 'interp "free id")]))
```